claude-windows / 7dc3da51-af38-47b6-83f6-fdda83b60664

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-inde
xer\7dc3da51-af38-47b6-83f6-fdda83b60664\subagents\workflows\wf_eb99bd9f-440\agent-
a0ca3716211c360ea.jsonl`
- SHA-256: `7c9dd3331b83c41f92883808958a9e12cf5c6027d52f159974d55198b48042af`
- Source modified: `2026-07-06T06:08:27+00:00`
- Imported at: `2026-07-08T16:00:12+00:00`
- project: `wf_eb99bd9f-440`
- session_id: `7dc3da51-af38-47b6-83f6-fdda83b60664`

Transcript

1. user (2026-07-06T06:04:56.586Z)

```
You are reviewing GitHub PR #35 of the repo at C:/Users/avidu/Projects/Telegram (branch
feat/track-b-domain-schema -> main).
It implements ADR 0011 (docs/adr/0011-provider-neutral-domain-schema.md): a provider-neutral
domain schema (Track B). Changed files ONLY: src/tg_video_filter/db.py,
src/tg_video_filter/models.py,
tests/test_db.py, tests/test_delivery.py. Diff: +1299/-150.

Ground rules:
- READ the actual code (Read/Grep). Cite exact file:line for every claim.
- You MAY run the venv python for a targeted repro:
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe <script>. The DB layer is aiosqlite;
  use db._apply_schema(conn) on an aiosqlite ":memory:" connection to build the schema, and
  db.upsert_account(...) to seed. A working repro is stronger evidence than prose.
- Compare against v1 behaviour on main when relevant: git -C C:/Users/avidu/Projects/Telegram
show main:src/tg_video_filter/db.py
- Authoritative local gate ALREADY RUN (do not re-run the full suite): ruff check clean; pytest
  434 passed @ 96.63% coverage (floor 80%); ruff format --check FAILS only on commands.py which
is
  NOT in this PR (pre-existing on main) ? do not report that as a PR finding.
- Report ONLY defects that ship in THIS PR's diff. No style nits already passing ruff. No
praise.
- Each finding needs a concrete failure scenario (inputs -> wrong result). Rank most-severe
first.
- Severity: blocker (data loss/corruption/wrong core behaviour), high (real bug, narrow
trigger),
  medium (latent/edge), low (hygiene). Be honest; downgrade if trigger is unrealistic.

ADVERSARIALLY VERIFY this single finding. Your job is to REFUTE it if you can ? read the exact
code at the
cited lines, trace the real control flow, and if useful run a repro with
C:/Users/avidu/Projects/Telegram/.venv/Scripts/python.exe. Only return CONFIRMED if
the defect genuinely ships in THIS PR's diff and produces the described wrong behaviour. Return
REFUTED if
the code actually handles it, the trigger cannot occur, or it's pre-existing (not in this diff).
Return
PLAUSIBLE only if you cannot fully settle it. Also return the corrected severity.

FINDING (dimension dedup):
title: Tier-2 fingerprint dedup is regressed: re-uploads with a new document_id are no longer
deduped
severity(claimed): high
file:line: src/tg_video_filter/db.py:1151
summary: get_video_by_content_fingerprint now looks up dedup_key='tg:fp:S:D:H:W', but any prior
```

video that had a document_id was stored under 'tg:doc:X', so the fingerprint lookup misses it and a re-upload (identical bytes, new document.id) is admitted as new ? losing ADR 0006 Tier-2 dedup for the overwhelmingly common case (nearly all Telegram videos carry a document_id).
failure_scenario: User previously saw video V in channel A: it had document_id=111, size=5000000, dur=60, h=1080, w=1920 -> stored in content_items with dedup_key='tg:doc:111'. Someone re-encodes/re-uploads the identical file to channel B; Telegram assigns a NEW document_id=222. telethon_client._on_message runs the tier chain: Tier-0 (channel B, new msg) misses; Tier-1 get_video_by_document_id(222) misses; Tier-2 get_video_by_content_fingerprint(5000000,60,1080,1920) computes/looks up dedup_key='tg:fp:5000000:60:1080:1920' and MISSES (the prior row is under 'tg:doc:111'). All tiers report 'new', insert_video(V') succeeds (dedup_key 'tg:doc:222' collides with nothing), and the DUPLICATE video is stored AND forwarded to the delivery target. On main, get_video_by_content_fingerprint did `SELECT ... WHERE size/dur/h/w match` against ALL videos regardless of document_id and would have found the prior row, returning early and suppressing the duplicate.
evidence: Repro on an in-memory DB: seed VideoRecord(document_id=99999,size=5000000,dur=60,h=1080,w=1920) -> stored dedup_key ['tg:doc:99999']; then get_video_by_content_fingerprint(5000000,60,1080,1920) returns None even though 1 row physically matches all four fingerprint columns, because the shim looks up key 'tg:fp:5000000:60:1080:1920' (db.py:1159-1173, compute_dedup_key branch order db.py:286-291). Full-chain repro: all three tiers miss and insert_video succeeds -> content_items ends with 2 rows (duplicate admitted/delivered). v1 behaviour: git show main:src/tg_video_filter/db.py get_video_by_content_fingerprint = `SELECT * FROM videos WHERE user_id=? AND size_bytes=? AND duration_s=? AND height=? AND width=? LIMIT 1` (matches ANY prior video incl. document-keyed). Caller telethon_client.py:207-236 uses this as Tier-2. README.md:31-32 advertises 'Never forwards the same video twice ? three-tier content dedup'; docs/design.md:224-225 defines Tier-2 as catching 're-upload of an identical file with a new document.id' ? the exact case now broken.
Root cause: dedup_key is a single-tier discriminator (doc XOR fp XOR msg), so a fp-shaped lookup can never see a doc-keyed row; the 3 tiers are mutually exclusive storage keys, not 3 independent read paths as v1 had.
recommendation: Make the Tier-2 shim query the fingerprint COLUMNS rather than the fp-shaped dedup_key, restoring v1 semantics: e.g. add get_content_item_by_fingerprint(account_id, size_bytes, duration_s, height, width) that does `SELECT * FROM content_items WHERE account_id=? AND size_bytes=? AND duration_s=? AND height=? AND width=? LIMIT 1` and have get_video_by_content_fingerprint delegate to it. (The dedup_key UNIQUE constraint can stay as-is; it never enforced fingerprint uniqueness on main either ? Tier-2 was always a read-path check.) Also add a test that seeds a prior video WITH a document_id and asserts the fingerprint lookup still hits ? the existing test_get_video_by_content_fingerprint_hit (tests/test_db.py:402) seeds a video with NO document_id, so the prior row lands under 'tg:fp:...' and the test passes while masking the regression.

2. assistant (2026-07-06T06:04:59.474Z)

I'll verify this finding by reading the actual code at the cited lines and tracing the control flow.

3. user (2026-07-06T06:05:00.141Z)

```
1130        src = await get_source_by_provider_id(conn, user_id, "telegram", str(channel_id))
1131        if src is None:
1132            return None
1133        item = await get_content_item_by_message(conn, user_id, src.id, str(message_id))
1134        return _content_item_to_video(item, src, user_id, channel_id) if item else None
1135
1136
1137    async def get_video_by_document_id(
1138        conn: aiosqlite.Connection, user_id: int, document_id: int
1139    ) -> VideoRecord | None:
1140        """Deprecated shim: Tier-1 lookup via ``compute_dedup_key`` + ``content_items``."""
1141        dedup_key = compute_dedup_key(provider="telegram", document_id=str(document_id))
1142        item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
1143        if item is None:
1144            return None
1145        src = await get_source_by_id(conn, item.source_id)
1146        if src is None:
```

```
            return None
        return _content_item_to_video(item, src, user_id, int(src.provider_source_id))


    async def get_video_by_content_fingerprint(
        conn: aiosqlite.Connection,
        user_id: int,
        size_bytes: int,
        duration_s: int,
        height: int,
        width: int,
    ) -> VideoRecord | None:
        """Deprecated shim: Tier-2 lookup via ``compute_dedup_key`` + ``content_items``."""
        dedup_key = compute_dedup_key(
            provider="telegram",
            size_bytes=size_bytes,
            duration_s=duration_s,
            height=height,
            width=width,
        )
        item = await get_content_item_by_dedup_key(conn, user_id, dedup_key)
        if item is None:
            return None
        src = await get_source_by_id(conn, item.source_id)
        if src is None:
            return None
        return _content_item_to_video(item, src, user_id, int(src.provider_source_id))


    async def get_source_by_id(conn: aiosqlite.Connection, source_id: int) -> Source | None:
        """Resolve a source by its surrogate PK (used by the legacy video shims)."""
        conn.row_factory = aiosqlite.Row
        async with conn.execute("SELECT * FROM sources WHERE id = ?", (source_id,)) as cur:
            row = await cur.fetchone()
        return _row_to_source(row) if row else None


    async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
        """Deprecated shim: insert into ``content_items`` (ADR 0011).

        Resolves the legacy ``channel_id`` to a :class:`Source` row (upserting it
        if absent), computes the provider-neutral ``dedup_key`` via
        :func:`compute_dedup_key`, and inserts a :class:`ContentItem`. The legacy
        ``VideoRecord`` shape is reconstructed from the inserted row.
        """
        src = await upsert_source(
            conn,
            Source(
                account_id=video.user_id,
                provider="telegram",
                provider_source_id=str(video.channel_id),
                kind="channel",
                watched=True,
```

4. user (2026-07-06T06:05:00.999Z)

```
        size_bytes: int | None = None,
        duration_s: int | None = None,
        height: int | None = None,
        width: int | None = None,
    ) -> str:
        """Compute the provider-neutral ``dedup_key`` for a ``ContentItem``.

        Implements the ADR 0011 Option 1A decision tree:

```

```
269         - **Telegram** (``provider == "telegram"``):
270           1. If ``document_id`` is set ? ``"tg:doc:{document_id}"``.
271           2. Else if **all four** fingerprint fields (size, duration, height, width)
272              are set ? ``"tg:fp:{size}:{dur}:{h}:{w}"``.
273           3. Else ? ``"tg:msg:{source_id}:{provider_message_id}"`` (Tier-0 fallback;
274              ``source_id`` is required because Telegram ``message_id`` is only
275              unique within a channel).
276         - **Other providers** (``provider != "telegram"``): fall back to the Tier-0
277           ``"{provider}:msg:{source_id}:{provider_message_id}"`` shape. Future
278           providers add their own branches (e.g. ``"wa:sha256:{blob_hash}"``) here.
279 
280         Raises ``ValueError`` if the fallback branch is reached without the fields
281         it requires (``source_id`` + ``provider_message_id``) ? a degenerate
282         ``dedup_key`` is worse than a loud failure at ingestion time.
283         """
284         p = provider.lower()
285         if p == "telegram":
286             if document_id is not None and document_id != "":
287                 return f"tg:doc:{document_id}"
288             # All four fingerprint fields are required to avoid a degenerate
289             # ``tg:fp:0:0:0:0`` key when some metadata is missing.
290             if None not in (size_bytes, duration_s, height, width):
291                 return f"tg:fp:{size_bytes}:{duration_s}:{height}:{width}"
292             # Tier-0 fallback.
293             if source_id is None or provider_message_id is None:
294                 raise ValueError(
295                     "Telegram dedup_key fallback requires source_id and provider_message_id
"
296                     "when document_id and the full fingerprint are unavailable"
297                 )
298             return f"tg:msg:{source_id}:{provider_message_id}"
299         # Future-proof default for providers without a dedicated branch.
300         if source_id is None or provider_message_id is None:
301             raise ValueError(
302                 f"dedup_key for provider={provider!r} requires source_id and
provider_message_id"
303             )
304         return f"{p}:msg:{source_id}:{provider_message_id}"
305 
306 
307     async def _apply_migrations(conn: aiosqlite.Connection) -> None:
308         """Run every registered migration whose version > current ``user_version``."""
309         async with conn.execute("PRAGMA user_version") as cur:
310             row = await cur.fetchone()
311         current = int(row[0]) if row else 0
312 
313         for version, fn in _MIGRATIONS:
314             if version <= current:
315                 continue
316             await fn(conn)
317             await conn.execute(f"PRAGMA user_version = {version}")
318 
319 
```

5. assistant (2026-07-06T06:05:06.915Z)

Now let me look at `get_content_item_by_dedup_key` and confirm the storage key for a video with a document_id. Let me also check the v1 behavior on main.

6. user (2026-07-06T06:05:08.559Z)

```
1008:async def get_content_item_by_dedup_key(
1009-    conn: aiosqlite.Connection, account_id: int, dedup_key: str
1010-) -> ContentItem | None:
1011-    """Provider-neutral dedup lookup by ``dedup_key`` (Option 1A).
```

```
1012-
1013-    Replaces the Tier-1 (``document_id``) and Tier-2 (fingerprint) lookups with
1014-    a single query: the ``dedup_key`` already encodes whichever tier was
1015-    applicable at ingestion time.
1016-    """
1017-    conn.row_factory = aiosqlite.Row
1018-    async with conn.execute(
1019-        "SELECT * FROM content_items WHERE account_id = ? AND dedup_key = ?",
1020-        (account_id, dedup_key),
```

7. user (2026-07-06T06:05:09.951Z)

```
640:async def get_video_by_content_fingerprint(
641-    conn: aiosqlite.Connection,
642-    user_id: int,
643-    size_bytes: int,
644-    duration_s: int,
645-    height: int,
646-    width: int,
647-) -> VideoRecord | None:
648-    """Tier-2 content dedup: metadata fingerprint match ? probably same video.
649-
650-    Catches re-uploads where Telegram assigns a new ``document.id`` but the
651-    file bytes are identical. Uses (size_bytes, duration_s, height, width)
652-    as a composite key ? the odds of two different videos sharing all four
653-    values are negligible for this use case.
654-    """
655-    conn.row_factory = aiosqlite.Row
```

8. assistant (2026-07-06T06:05:13.778Z)

```
The finding is well-documented. Let me confirm the v1 SQL query and the caller (telethon_client)
tier chain to verify this is really the Tier-2 read path.
```

9. user (2026-07-06T06:05:15.386Z)

```
    conn.row_factory = aiosqlite.Row
    async with conn.execute(
        """SELECT * FROM videos
           WHERE user_id = ?
             AND size_bytes = ?
             AND duration_s = ?
             AND height = ?
             AND width = ?
           LIMIT 1""",
        (user_id, size_bytes, duration_s, height, width),
    ) as cur:
        row = await cur.fetchone()
    return _row_to_video(row) if row else None


async def insert_video(conn: aiosqlite.Connection, video: VideoRecord) -> VideoRecord:
    """Insert a new video row. Raises IntegrityError on a dedup-key collision.

    Two dedup constraints protect against duplicates:
      - ``UNIQUE (user_id, channel_id, message_id)`` ? same message instance.
      - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
```

10. user (2026-07-06T06:05:15.899Z)

```
193:            existing = await db.get_video_by_document_id(conn, user.id, document_id)
214:            existing = await db.get_video_by_content_fingerprint(
```

11. assistant (2026-07-06T06:05:20.304Z)

The v1 query matches ANY video by fingerprint columns regardless of document_id. The caller uses it as Tier-2 after Tier-1 (document_id) misses. Now let me confirm the caller's tier chain and run the repro.

12. user (2026-07-06T06:05:21.182Z)

```
185
186             # Tier 0: same (user, channel, message) instance ? already processed.
187             existing = await db.get_video(conn, user.id, channel_id, message_id)
188             if existing is not None:
189                 return
190
191             # Tier 1: same Telegram document.id ? forwarded/cross-posted file.
192             if document_id is not None:
193                 existing = await db.get_video_by_document_id(conn, user.id, document_id)
194                 if existing is not None:
195                     log.info(
196                         "video_dedup_document_id user=%s doc_id=%s channel=%s msg=%s "
197                         "original_channel=%s original_msg=%s",
198                         user.id,
199                         document_id,
200                         channel_id,
201                         message_id,
202                         existing.channel_id,
203                         existing.message_id,
204                     )
205                     return
206
207             # Tier 2: metadata fingerprint ? same file re-uploaded (rare but possible).
208             if (
209                 meta.size_bytes is not None
210                 and meta.duration_s is not None
211                 and meta.height is not None
212                 and meta.width is not None
213             ):
214                 existing = await db.get_video_by_content_fingerprint(
215                     conn,
216                     user.id,
217                     meta.size_bytes,
218                     meta.duration_s,
219                     meta.height,
220                     meta.width,
221                 )
222                 if existing is not None:
223                     log.info(
224                         "video_dedup_fingerprint user=%s size=%s dur=%s h=%s w=%s "
225                         "channel=%s msg=%s original_channel=%s original_msg=%s",
226                         user.id,
227                         meta.size_bytes,
228                         meta.duration_s,
229                         meta.height,
230                         meta.width,
231                         channel_id,
232                         message_id,
233                         existing.channel_id,
234                         existing.message_id,
235                     )
236                     return
237
238             # Lazily record the channel (best-effort; failures are non-fatal).
239             try:
```

13. assistant (2026-07-06T06:05:24.514Z)

The control flow confirms the finding exactly: Tier-1 (`get_video_by_document_id`) checks

`tg:doc:222` (misses), then Tier-2 (`get_video_by_content_fingerprint`) looks up `tg:fp:...` which will miss because the prior row is stored under `tg:doc:111`. Let me run the repro to be certain.

14. user (2026-07-06T06:07:34.313Z)

Command running in background with ID: bg1fs8e89. Output is being written to: C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\7dc3da51-af38-47b6-83f6-fdda83b60664\tasks\bg1fs8e89.output. You will be notified when it completes. To check interim output, use Read on that file path.

15. user (2026-07-06T06:07:42.781Z)

```
Traceback (most recent call last):
  File "C:\Users\avidu\AppData\Local\Temp\repro.py", line 26, in <module>
    asyncio.run(main())
    ~~~~~~~~~~~^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 195, in run
    return runner.run(main)
           ~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\runners.py", line 118, in run
    return self._loop.run_until_complete(task)
           ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\asyncio\base_events.py", line 725, in run_until_complete
    return future.result()
           ~~~~~~~~~~~~~^^
  File "C:\Users\avidu\AppData\Local\Temp\repro.py", line 8, in main
    await db.upsert_account(conn, 1, "u")
          ~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^
TypeError: upsert_account() takes 2 positional arguments but 3 were given
```

16. assistant (2026-07-06T06:07:47.239Z)

Let me check the signatures for `upsert_account` and `VideoRecord`.

17. user (2026-07-06T06:07:48.901Z)

```
730:async def upsert_account(conn: aiosqlite.Connection, account: Account) -> Account:
731-    """Insert or update an account row; returns the persisted Account."""
732-    await conn.execute(
733-        """INSERT INTO accounts (id, telegram_id, provider, display_name, created_at)
734-           VALUES (?, ?, ?, ?, ?)
735-           ON CONFLICT(id) DO UPDATE SET
736-             telegram_id=excluded.telegram_id,
```

18. user (2026-07-06T06:07:49.551Z)

```
295:class VideoRecord(BaseModel):
296-    """A seen video message + its metadata. Drives dedup and future /search.
297-
298-    Deprecated: a provider-neutral view over :class:`ContentItem` (ADR 0011).
299-    New code should use :class:`ContentItem`; the ``db`` layer maps
300-    ``VideoRecord`` ? ``ContentItem`` so existing callers and tests keep working.
301-    """
302-
303-    model_config = ConfigDict(frozen=True)
304-
305-    id: int | None = Field(default=None, description="Surrogate PK (DB-assigned)")
306-    user_id: int
307-    channel_id: int = Field(description="Origin Telegram channel id")
308-    message_id: int = Field(description="Telegram message id within the channel")
```

```
309-    document_id: int | None = Field(
310-        default=None,
311-        description="Telegram document.id ? reused across forwards; Tier-1 content dedup",
312-    )
313-
```

19. user (2026-07-06T06:07:52.414Z)

```
35:class Account(BaseModel):
36-    """A product identity ? the owner of config, channels, routes, and content.
37-
38-    Decoupled from any specific provider's session (ADR 0010). ``telegram_id``
39-    is the bridge used by the Bot API front-door (``from.id``) and by the
40-    legacy userbot path; it is nullable so future providers that don't have a
41-    Telegram id (e.g. WhatsApp) can still own an ``Account``.
42-    """
43-
44-    model_config = ConfigDict(frozen=True)
45-
46-    id: int = Field(description="Stable surrogate PK; the owner key for all downstream
tables")
47-    telegram_id: int | None = Field(
```

20. user (2026-07-06T06:08:04.360Z)

```
stored keys: ['tg:doc:111']
fingerprint lookup result: None
doc_id 222 lookup: None
```

21. assistant (2026-07-06T06:08:08.328Z)

Repro confirms it exactly. The prior video with `document_id=111` is stored under `tg:doc:111`. The Tier-2 fingerprint lookup (identical size/dur/h/w) returns `None` because it queries `tg:fp:5000000:60:1080:1920` which does not exist. Both Tier-1 (doc 222) and Tier-2 miss, so the re-upload with a new document_id would be admitted as new and forwarded.

Let me verify the v1 behavior would have caught it, confirming the regression.

22. user (2026-07-06T06:08:10.339Z)

```
171:        "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
317:        "ON videos(user_id, document_id) WHERE document_id IS NOT NULL"
598:        "SELECT * FROM channels WHERE user_id = ? AND channel_id = ?",
615:           WHERE user_id = ? AND channel_id = ? AND message_id = ?""",
633:           WHERE user_id = ? AND document_id = ?""",
658:           WHERE user_id = ?
675:      - ``UNIQUE (user_id, document_id) WHERE document_id IS NOT NULL`` ?
715:        "SELECT config_json FROM filter_config WHERE user_id = ?", (user_id,)
742:        "SELECT * FROM videos WHERE user_id = ? ORDER BY id DESC LIMIT ?",
```

23. assistant (2026-07-06T06:08:18.746Z)

Confirmed. On main, the v1 `get_video_by_content_fingerprint` (lines 656-666) queries `videos WHERE user_id=? AND size_bytes=? AND duration_s=? AND height=? AND width=?` with no `document_id` filter ? it matches ANY prior video including document-keyed ones. So on main, the same scenario would find the prior row and suppress the duplicate.

The regression is real, ships in this PR's diff (db.py:1151-1173, the new shim), and produces the exact wrong behavior described: an identical-bytes re-upload with a new document_id is admitted as new and forwarded, defeating ADR 0006 Tier-2 dedup for the common case where videos carry a document_id.

On severity: the claimed "high" is appropriate. It's a real bug with a realistic trigger (nearly all Telegram videos carry a document_id, and cross-posted re-encodes/re-uploads are exactly what

Tier-2 exists to catch), the result is a user-visible duplicate forward (the product's headline promise: "never forwards the same video twice"), but it's narrow in that it requires a genuine re-upload with a fresh document_id rather than a plain forward (forwards preserve document_id and are still caught by Tier-1). Not a blocker since it's a dedup gap, not data corruption of existing rows.

24. user (2026-07-06T06:08:27.501Z)

Structured output provided successfully